DO SIMULADOR AO SILÍCIO · ENCONTRO 6 · LIGA PROTOFPGA

Folha de atividade

Nomes da dupla: _____ Data: ____ / ____

Parte A — Atividade em aula (em duplas)

Trabalhem na mesma máquina, com o MARS aberto. Toda a atividade é feita no MARS; a placa entra na demonstração, no fim. A folha de consulta, nas últimas páginas, tem as 37 instruções do MIPS_S, a tabela de equivalências e os endereços de entrada e saída. Cada parte tem tempo curto. Se travarem, chamem.

A1 — Caçar (5 min)

1. Abram o `aula4_laco.asm` (do encontro 4) e cliquem em Assemble. Na janela Text Segment, percorram a coluna **Basic** e anotem as instruções que **não** existem no MIPS_S:

Instrução	Quantas vezes aparece	O que pôr no lugar

2. O programa usa `li`, `la` e `move`. Em que instruções o MARS transformou `li $s1, 8` e `move $t2, $t4`? Elas existem no MIPS_S?

A2 — Prever (8 min)

Abram o `aula6_contador_mars.asm`. É o contador da demonstração, com as funções de entrada e saída trocadas por teclado e Run I/O: em vez de apertar um botão, digita-se o seu código (C = 1, U = 2, L = 4, R = 8, D = 16; 0 encerra). **Antes de executar**, leiam o laço principal e prevejam os LEDs depois de cada passo. O contador começa em 0.

3. Preencham a coluna "Previsão" com os 16 bits, do LED15 ao LED0:

	Botão (código)	Chaves	Previsão dos LEDs
a	`BTNC (1)`	`5`	□□□□ □□□□ □□□□ □□□□
b	`BTNL (4)`	—	□□□□ □□□□ □□□□ □□□□
c	`BTNU (2)`	—	□□□□ □□□□ □□□□ □□□□
d	`BTND (16)`	—	□□□□ □□□□ □□□□ □□□□
e	`BTND (16)`	—	□□□□ □□□□ □□□□ □□□□
f	`BTNR (8)`	—	□□□□ □□□□ □□□□ □□□□
g	`BTNC (1)`	`0`	□□□□ □□□□ □□□□ □□□□
h	`BTND (16)`	—	□□□□ □□□□ □□□□ □□□□

4. Agora executem (Run) e digitem a sequência; no fim, digitem 0. Conferiram? Se algum passo deu diferente, qual, e por quê?

5. No passo h, os 16 LEDs acendem. De onde veio esse número? O que acontece se, em seguida, vocês apertarem BTNU?

A3 — Modificar (7 min)

6. O MIPS_S não tem `mul`. No `aula6_contador_mars.asm`, troquem a linha `srl $s0, $s0, 1` (o ÷ 2 do BTNR) por instruções que **multipliquem o contador por 3**, usando só instruções do MIPS_S. Pista: 3x = x + 2x. Escrevam aqui o que vocês puseram:

7. Testem: BTNC com as chaves em 5 e depois BTNR duas vezes. Que valores apareceram, em decimal?

`depois do 1º BTNR: _____` `depois do 2º BTNR: _____`

8. Daria para usar `multu` e `mflo`? Comparem as duas soluções em número de ciclos do MIPS_S.

A4 — Portar (10 min)

9. Façam o programa do encontro 4 rodar no MIPS_S. Copiem o `aula4_laco.asm` para um arquivo novo e mudem o que for preciso para que ele: (a) use só instruções do MIPS_S; (b) mostre a **soma** nos LEDs, em vez de imprimir; (c) termine em `fim: j fim`, em vez do serviço 10. Anotem as linhas que vocês mudaram:

10. Confiram a coluna Basic. Depois executem com Run e, um segundo depois, cliquem em Stop (o programa não termina sozinho). Na janela Data Segment, escolham `0xffff0000 (MMIO)` no menu de baixo. Que valor está no endereço `0xffff0008`?

`Mem[0xffff0008] = _____   (hexadecimal)`

11. Na placa, quais LEDs vão acender? Pintem as bolinhas:

```
LED   15 14 13 12  11 10  9  8   7  6  5  4   3  2  1  0

       ○  ○  ○  ○   ○  ○  ○  ○   ○  ○  ○  ○   ○  ○  ○  ○
```

Parte B — Desafio (opcional)

Enunciado: uma calculadora de 16 bits

Partindo do `aula6_contador_mars.asm`, escreva uma calculadora. O valor mostrado nos LEDs é um acumulador, que começa em 0. A cada botão, o programa lê as chaves e faz uma conta:

BTNC — acumulador = chaves. **BTNU** — acumulador + chaves. **BTND** — acumulador − chaves.

BTNL — acumulador × chaves. **BTNR** — resto da divisão do acumulador pelas chaves (se as chaves valerem 0, não faz nada).

Use só instruções do MIPS_S: para multiplicar e dividir, `multu`, `divu`, `mflo` e `mfhi`. Mantenha só os 16 bits de baixo, como no contador.

Teste com: C 100 → 100; U 23 → 123; L 2 → 246; R 7 → 1; D 2 → 65535 (todos os LEDs acesos).

Pergunta extra: o código do botão chega em `$v0`, mas a função `le_chaves` também devolve em `$v0`. Onde você guardou o botão enquanto lia as chaves? Por quê?

Espaço para o seu programa:

Parte C — Exercícios de revisão

Onze perguntas, para fazer em casa. As duas últimas pedem o MARS. Tentem responder tudo antes de abrir o gabarito.

1. O MIPS_S tem `addu`, mas não tem `add`. O resultado das duas é diferente? O que se perde ao trocar uma pela outra, e por que o MIPS_S não precisa de `add`?

2. Reescreva cada linha usando só instruções do MIPS_S: (a) `addi $t0, $t0, -1`; (b) `sub $t2, $t0, $t1`; (c) `mul $t3, $t1, $t2`.

3. Quais destas pseudoinstruções podem ser usadas num programa para o MIPS_S? Justifique com as instruções em que o MARS as transforma: `li $t0, 5`, `move $t1, $t0`, `blt $t0, $t1, L`, `neg $t2, $t0`, `subi $t3, $t0, 1`.

4. O laço abaixo, da apresentação do MIPS_S, soma uma constante a cada elemento de um vetor de 11 posições. (a) Quantos ciclos gasta uma volta? (b) A inicialização, antes do laço, gasta 34 ciclos, e a saída (o último `blez`, que desvia, e o `j end`), 8. Quantos ciclos ao todo? (c) Quanto tempo, a 50 MHz?

```
loop: blez  $t1, end
      lw    $t3, 0($t0)
      addu  $t3, $t3, $t2
      sw    $t3, 0($t0)
      addiu $t0, $t0, 4
      addiu $t1, $t1, -1
      j     loop
end:  j     end
```

5. Quanto tempo leva um `multu` a 50 MHz? Quantas instruções `addu` caberiam nesse mesmo tempo?

6. Escreva as instruções que acendem só o LED15 e o LED0, e nenhum outro.

7. Escreva um programa que fique, para sempre, copiando as chaves para os LEDs: cada chave levantada acende o LED de mesmo número. Quantas instruções ele tem?

8. Um colega diz: "o processador tem uma instrução especial para acender os LEDs, a `sw`". O que está errado nessa frase? Quem decide que um `sw` em `0xFFFF0008` acende LEDs?

9. Liste tudo o que precisa mudar no `aula5_fatorial.asm` (encontro 5) para ele rodar no MIPS_S e mostrar o resultado, 120, nos LEDs. Quais LEDs acendem?

10. Num programa para a placa, por que não se usa o serviço 10 para terminar? O que o processador fica fazendo depois de chegar em `fim: j fim`?

11. No MARS: a `espera_botao` espera cerca de 42 ms com 262 144 voltas. Quantas voltas seriam necessárias para esperar 100 ms a 50 MHz? Esse número cabe num imediato de 16 bits? Escreva as instruções que carregam esse número em `$t1` e confira o valor no MARS.

DO SIMULADOR AO SILÍCIO · ENCONTRO 6 · LIGA PROTOFPGA

Folha de consulta

Tudo o que a atividade e os exercícios pedem. Verde é entrada e saída; vermelho, o que não existe no MIPS_S; âmbar, endereço; rosa, desvio.

As 37 instruções do MIPS_S

Grupo	Instruções	Ciclos
Aritméticas	`addu subu addiu multu divu`	4 (`multu`, `divu`: 67)
Lógicas	`and or xor nor andi ori xori`	4
Deslocamento	`sll sllv sra srav srl srlv`	4
Memória	`lw sw lbu sb`	`lw`, `lbu`: 5; `sw`, `sb`: 4
Comparação	`slt sltu slti sltiu`	4
Desvio e salto	`beq bne bgez blez j jal jr jalr`	4
Outras	`lui mfhi mflo`	4

Valem também `li`, `la` e `move`, que o MARS troca por `addiu`, `lui` e `ori`, e `addu`. Na dúvida, confira a coluna **Basic**.

Tabela de equivalências

Em vez de	No MIPS_S	Atenção
`add $d, $s, $t`	`addu $d, $s, $t`	transbordo passa sem aviso
`addi $d, $s, k`	`addiu $d, $s, k`	idem
`sub $d, $s, $t`	`subu $d, $s, $t`	idem
`mul $d, $s, $t`	`multu $s, $t` e `mflo $d`	32 bits de baixo iguais
`div $s, $t`	`divu $s, $t`; `mflo` quociente, `mfhi` resto	só números ≥ 0
`lb $d, k($s)`	`lbu $d, k($s)`	não estende o sinal
`syscall`	`lw` / `sw` nos endereços de E/S	veja abaixo
(o `$sp` do MARS)	`li $sp, 0x10014000`	se usar a pilha

Entrada e saída mapeadas em memória

Endereço	Acesso	O que é	Uso
`0xFFFF0000`	leitura	botões, bits 4–0	`lw $t0, 0($s7)`
`0xFFFF0004`	leitura	chaves, bits 15–0	`lw $v0, 4($s7)`
`0xFFFF0008`	escrita	LEDs, bits 15–0	`sw $a0, 8($s7)`

Base: `lui $s7, 0xFFFF`. Máscaras dos botões, para usar com `andi`: **C** `0x01` · **U** `0x02` · **L** `0x04` · **R** `0x08` · **D** `0x10`.

Mapa de memória do MIPS_S na placa

Região	Endereços
Programa (2 KB)	`0x00400000` a `0x004007FF`
Dados (16 KB)	`0x10010000` a `0x10013FFF`
Pilha	começa em `0x10014000` e cresce para baixo
Periféricos	`0xFFFF0000` a `0xFFFFFFFF`

As funções de E/S do contador

Função	Na placa	No MARS (versão _mars)
`espera_botao`	espera um botão; código em `$v0`	lê o código digitado: C = 1, U = 2, L = 4, R = 8, D = 16; 0 encerra
`le_chaves`	`lw $v0, 4($s7)`	lê um número de 0 a 65535
`mostra_leds`	`sw $a0, 8($s7)`	imprime os 16 bits de `$a0`, do LED15 ao LED0

Tempo

Clock	50 MHz: um ciclo dura **20 ns**
Tempo	ciclos × 20 ns (ou ciclos ÷ 50 000 000 segundos)
Laço de espera	`addiu` + `bne` = 8 ciclos por volta

Ver a E/S no MARS

Janela Data Segment → menu de baixo → `0xffff0000 (MMIO)`. A palavra em `0xffff0008` é o que os LEDs mostrariam.

Hexadecimal ↔ binário

0 0000	**1** 0001	**2** 0010	**3** 0011	**4** 0100	**5** 0101	**6** 0110	**7** 0111
8 1000	**9** 1001	**A** 1010	**B** 1011	**C** 1100	**D** 1101	**E** 1110	**F** 1111